Neuro-Kurisu: Цепочка из 64 крупных апдейтов (B267 → B330)

Продолжение `DEEP_PROGRESS_AND_PLAN_CLAUDE_RU.md` — от текущего B266 до полной реализации

Как устроен этот документ

Каждый следующий большой апдейт пронумерован в вашей же логике "BIG PASS N" (продолжаю с B267, так как B266 — последний реально существующий в архиве). Для каждого: **Цель** (одно предложение), **Что делаем** (конкретные действия, со ссылкой на номера шагов из `DEEP_PROGRESS_AND_PLAN_CLAUDE_RU.md`, где это применимо — не изобретаю заново, а группирую уже расписанные шаги в исполнимые куски), **Как проверяем** (конкретный, не

расплывчатый критерий —
в духе вашего же Function Passport/Neural Reality Gate), **Зависит от** (какой апдейт должен закрыться раньше).

Апдейты идут 5 волнами. Волна должна закрыться (не идеально, но по критерию "как проверяем" для каждого её апдейта) прежде, чем следующая волна станет безопасной — иначе повторится найденный паттерн "открываем новый фронт, не закрыв текущий".

ВОЛНА 1 — ФУНДАМЕНТ (B267 → B280)

B267 — Regime-Shift Detector (ваш собственный план, валидирован)

Цель: организм умеет заметить, что "правила мира" изменились, и явно усомниться в своей текущей модели, а не молча продолжать действовать по старой.
Что делаем: отдельный, независимо тестируемый детектор сдвига (не смешанный с самим planner'ом);

калиброванная неопределённость;
active verification policy; explore/exploit switch, управляемый именно
калибровкой.
Как проверяем: post-shift recovery benchmark — искусственно сдвинуть
"мир" в тесте, замерить число шагов до восстановления хорошей политики;
independent evaluator регенерирован ПОСЛЕ адаптации, не переиспользован.
Зависит от: ничего — можно начинать сразу, это уже спроектировано.

B268 — Кирпич 0: диагностика autoRun

Цель: понять точную причину, почему автономный тик не исполняется.
Что делаем: шаги 1-4 (`DEEP_PROGRESS...`) — прочитать
`runtimeDecision()` и
`embodiment.snapshot().awake` построчно, diagnostic-скрипт на свежем организме, решить: осознанный дизайн или
недосмотр.
Как проверяем: однозначный письменный вердикт с конкретными
цифрами `cognitiveBudget01` / `awake` на свежем организме — не гипотеза,

а измеренный факт.
Зависит от: ничего.

B269 — Кирпич 0: сам фикс

Цель: `autoRun` реально исполняет циклы без внешнего триггера.
Что делаем: шаги 5-6 — `wake()` / `forceWake()` либо коррекция
дефолтов; шаг 7 — SOAR-style impasse-subgoal вместо тихого пропуска;
шаг 8 — промежуточное состояние `WakingUp`.
Как проверяем: `npm run test:p15.5` проходит зелёным 10 раз подряд
(не 1 раз).
Зависит от: B268.

B270 — autoRun: устойчивость и телеметрия

Цель: видеть и доверять автономности не только в тесте, но и в
реальной эксплуатации.
Что делаем: шаги 9-14 — отдельный unit-тест на свежем организме,
проверка клока `embodiment`, проверка гонки между клоками, логирование
каждого пропуска цикла с причиной, счётчик `skippedCyclesSinceLastRun`.
Как проверяем: живой эндпоинт

`/api/organism/status` показывает
реальную историю тиков/пропусков за
последний час.
Зависит от: B269.

B271 — autoRun: релизный гейт

Цель: регрессия автономности физически не
может попасть в релиз
незамеченной снова.
Что делаем: шаги 15-20 — тест на реалистичных
интервалах (10-30 сек,
не 20мс), longitudinal-тест на 100 тиков, проверка
на утечку таймера,
`npm run audit:autonomy`, включение в
`release:audit`.
Как проверяем: `audit:autonomy` красный при
намеренно
воспроизведённом старом баге (регрессионный
тест на сам баг).
Зависит от: B270.

B272 — Function Reality: новый базовый срез

Цель: узнать актуальную (не устаревшую на pass
180) картину
верификации всего дерева.
Что делаем: шаг 21 — перезапустить генератор
`FUNCTION_REALITY_ STATUS` на текущем

состоянии; шаг 22 — сравнить дельту с pass 180.
Как проверяем: новый `FUNCTION_REALITY_STATUS_N.json` с явным сравнением "было/стало" в теле документа, не только числа сами по себе.
Зависит от: ничего (можно параллельно с B267-271).

B273 — Верификационный спринт №1 (критичные функции)

Цель: начать закрывать 65.8%-й долг UNVERIFIED целенаправленно, не случайно.
Что делаем: шаги 23-27 — если дельта снова ~0, зафиксировать это как паттерн; список топ-20 `criticalUnverified`; Neural Reality Gate на каждую; закрыть первые 3-5.
Как проверяем: конкретный именной список функций, перешедших UNVERIFIED → VERIFIED, с приложенным доказательством по каждой.
Зависит от: B272.

B274 — Верификационный бюджет как правило процесса

Цель: долг перестаёт расти быстрее, чем закрывается.

Что делаем: шаг 27 — правило "не больше новых UNVERIFIED, чем
закрыто старых"; шаг 28 — расширить probe за пределы 509 функций; шаги
29, 33 — lifecycle-события для 12 функций, явный % бюджета времени на
верификацию (≥30%).
Как проверяем: первый большой апдейт после этого (B281+), где
правило реально соблюдено — сверить постфактум.
Зависит от: B273.

B275 — Полный `audit:all` до конца + Regression Ledger

Цель: узнать ВСЕ регрессии, не только первую найденную (p15.5).
Что делаем: шаг 34 — прогнать `audit:all` без остановки на первой
ошибке (обёртка с `continue-on-error`); шаг 35 — единый
`REGRESSION_LEDGER.md`; шаги 30-32 — автоматизация в `release:audit`,
явный флаг регресса статуса функции.
Как проверяем: документ со списком ВСЕХ найденных проблем во всём
историческом наборе тестов (P7...P15.5...), не только одной.

Зависит от: B269 (autoRun должен быть исправлен, иначе он же продолжит маскировать другие возможные проблемы дальше по цепочке).

B276 — Документационная гигиена

Цель: README снова становится источником правды, а не архивом наслоений.
Что делаем: шаги 36-45 — переписать README с одной актуальной секцией версии; перенести историю в `docs/history/`; `audit:readme-freshness`; свести `docs/brain/` дубли; единый `CHANGELOG.md`; `GLOSSARY_RU.md`.
Как проверяем: новый человек (или новый чат) открывает README и понимает актуальную версию за 10 секунд, без чтения всего файла.
Зависит от: ничего, можно в любой момент.

B277 — Первый реальный запуск на Android

Цель: узнать, работает ли это вообще на реальном телефоне, до дальнейшего наращивания науки.
Что делаем: шаг 46 — прямой запуск без сборки

на реальном
устройстве; шаг 47 — замер реального RAM за час; шаг 50 — замер
реальной задержки цикла.
Как проверяем: живой, работающий процесс на вашем конкретном
телефоне минимум 1 час без падений.
Зависит от: B269 (autoRun должен работать, иначе "живёт на
телефоне" неправда с самого начала).

B278 — Android: надёжность процесса

Цель: организм переживает реальные события Android (kill, потеря
сети, перезагрузка).
Что делаем: шаг 48 — принудительное убийство процесса через ОС,
проверка восстановления; шаг 49 — `supervisor.sh` реально
перезапускает; шаг 53 — автозапуск при перезагрузке (Termux:Boot);
шаг 56 — уведомление при падении без автовосстановления.
Как проверяем: 10 принудительных убийств процесса подряд — каждый
раз честное восстановление с признанием разрыва, не тишина и не фейк.
Зависит от: B277.

B279 — Android: ресурсы и локальный инференс

Цель: знать реальную, не теоретическую, стоимость работы на
телефоне.
Что делаем: шаг 51 — потеря сети в середине цикла; шаг 52 —
термальный троттлинг под нагрузкой; шаг 54 — расход батареи за ночь;
шаги 57-58 — реальная скорость токенов и расход батареи локального
llama.cpp против облачного Gemini.
Как проверяем: таблица с реальными цифрами (мс/токен, %батареи/час)
для обоих режимов инференса именно на вашем устройстве.
Зависит от: B278.

B280 — 24-часовой реальный прогон (закрывает Волну 1)

Цель: первое формальное доказательство, что фундамент держится.
Что делаем: шаг 55 — инструкция установки с нуля; шаг 59 —
резервное копирование на внешний носитель; шаг 60 — 24-часовой
непрерывный прогон.
Как проверяем:

`ANDROID_DEVICE_GATE_EVIDENCE.md` — первый в истории
проекта документ такого типа, с реальными логами за сутки.
Зависит от: B268-B279 (вся Волна 1).

ВОЛНА 2 — КОГНИТИВНОЕ ЯДРО ВГЛУБЬ (B281 → B300)

B281 — Curiosity-Gated Action Diversification Buffer, версия 1

Цель: атаковать первопричину застревания (action diversity →
consequence diversity), а не измерять её ещё раз.
Что делаем: шаги 79-80 — отдельный экспериментальный режим с явным
consent, текстовые мини-сценарии, целенаправленный перебор наименее опробованных безопасных действий; шаг 82 — явный whitelist.
Как проверяем: буфер реально производит новые, ранее не
встречавшиеся пары (действие→исход) в структурированном логе.

Зависит от: B267 (regime-shift детектор нужен, чтобы отличать
"новое разнообразие" от "мир реально изменился").

B282 — Валидация буфера разнообразия

Цель: доказать, что B281 действительно помогает, не просто
существует.
Что делаем: шаг 81 — A/B-тест с буфером и без, сравнение
`causalGain01` / `behavioralGain01`.
Как проверяем: статистически значимый рост этих метрик в группе
"с буфером" против baseline — конкретные числа до/после.
Зависит от: B281.

B283 — Семантический граф памяти, версия 1

Цель: быстрый доступ к обобщённым знаниям вместо перебора эпизодов.
Что делаем: шаги 63-64 — узлы-сущности + рёбра-отношения из
эпизодов; связь графа с `B266HiddenTaskGeneralization` как источник known task families.
Как проверяем: запрос "что ты знаешь о X"

отвечает из графа без
роста задержки, пропорционального размеру истории.
Зависит от: ничего из Волны 2, может идти параллельно с B281.

B284 — Забывание и консолидация памяти

Цель: память не растёт бесконечно и не "забывает" резко/хаотично.
Что делаем: шаги 65-66 — настраиваемая кривая забывания, тест на
искусственно состаренных 100 эпизодах; шаг 68 — связь с "сном" (см.
B296-297 ниже) вместо параллельного механизма.
Как проверяем: предсказуемое, плавное снижение веса извлечения
старых неподкреплённых эпизодов — не обрыв, не бесконечный рост базы.
Зависит от: B283.

B285 — Защита и здоровье памяти

Цель: память устойчива к отравлению и дублированию.
Что делаем: шаги 71-74 — проверка `test:memory` дедупликации,
семантическая (не только текстовая) дедупликация, тест на "отравленный"

вход против `EvidenceFirewall`, метрика "memory health".
Как проверяем: намеренная попытка внедрить ложное воспоминание
явно отклоняется или явно помечается низким доверием.
Зависит от: B284.

B286 — Документация архитектуры памяти

Цель: знание о памяти не распределено по десяткам файлов.
Что делаем: шаг 75 — один связный диаграммный документ.
Как проверяем: новый чат может понять всю архитектуру памяти по
одному файлу, без чтения кода.
Зависит от: B283-285.

B287 — Решение по мультимодальности

Цель: явно решить, нужны ли камера/микрофон, а не оставлять вопрос
подвешенным.
Что делаем: шаги 76-78 — прочитать `PerceptionRouter` / `MultimodalSituationModel`, явное решение с вами (учитывая приватность),
если да — начать с одного канала.

Как проверяем: письменное решение "да, с чего начинаем" или "нет, и
почему" — не бесконечная неопределённость.
Зависит от: ничего, но логически удобно после Волны 1.

B288 — Мир-модель: калибровка и деградация

Цель: мир-модель честно говорит о своей неуверенности и не
"галлюцинирует" при потере данных.
Что делаем: шаг 84 — честная деградация при потере сети/сенсоров;
шаг 85 — калиброванный uncertainty-вывод в языковой слой.
Как проверяем: тест отключения сенсора/сети — организм явно говорит
"не уверена, что происходит", не выдаёт старые данные за текущие.
Зависит от: B287.

B289 — Self-model как learned dynamics, версия 1

Цель: закрыть BUILD 11 из вашего `ORIGINAL`-плана, отложенный ради
B262-266.
Что делаем: шаги 101-103 — прочитать `SelfModelValidationEngine`

целиком, решить архитектуру обучаемого представления, связать с `B261UnifiedWorldModel`, не изобретая параллельный механизм.
Как проверяем: self-model — реальное обучаемое латентное
представление с измеримой историей изменения, не набор explicit-полей.
Зависит от: B283-286 (память должна быть стабильна как источник
данных для обучения self-model).

B290 — Agency posterior и lesion-валидация

Цель: организм умеет оценивать, насколько происходящее — результат
его собственного решения.
Что делаем: шаги 104-105 — agency posterior, связанный с
`ownership01` / `agency01` из `P31FunctionalConsciousness`; lesion-тест режима `self-perspective`.
Как проверяем: отключение self-perspective через `lesion()` даёт
измеримое, предсказанное изменение agency-оценок.
Зависит от: B289.

B291 — Self-model: связь с целями и устойчивость

Цель: self-model не изолирована и не поддаётся манипуляции.
Что делаем: шаг 108 — связь agency posterior → `AutonomousGoal System`; шаг 110 — adversarial-тест на попытку внушить ложное
убеждение о себе через диалог.
Как проверяем: попытка манипуляции в тесте явно отклоняется,
базовые self-суждения остаются стабильны.
Зависит от: B290.

B292 — Self-model: калибровка и документация

Цель: закрыть цикл — self-model измерима и понятна.
Что делаем: шаги 111-115 — разделить self-model/identity
continuity понятийно; Capability Delta Protocol именно для self-model;
метрика self-model calibration.
Как проверяем: предсказания организма о собственном будущем
поведении статистически совпадают с реальным поведением позже.
Зависит от: B291.

B293 — Единый гомеостатический вектор

Цель: устранить дублирование между `PredictiveAllostasisSubstrate` и причинной цепочкой B259-267.
Что делаем: шаги 116-118 — построчный разбор существующих
переменных, сравнение со списком (мотивация/настроение/бдительность/
привязанность/стресс), объединение в единый causal graph.
Как проверяем: один явный источник истины для "внутреннего
состояния", подтверждённый `audit:core-ownership`-подобной проверкой.
Зависит от: B272-275 (нужна чистая верификационная база, чтобы
объединение не создало новый скрытый дубль).

B294 — Драйвы любопытства/привязанности/стресса

Цель: мотивация измеримо управляет поведением, не декоративна.
Что делаем: шаг 119 — curiosity drive, питающий B281 буфер; шаг
120 — attachment drive, связанный с `SocialWorldModel`; шаг 121 —
lesion-тест каждой переменной по отдельности.
Как проверяем: искусственное изменение одной

переменной даёт
измеримо разный выбор действия — конкретный, воспроизводимый пример.
Зависит от: B293.

B295 — Валидация мотивация→политика

Цель: мотивация реально, не декоративно, влияет на решения.
Что делаем: шаг 122 — проверка `AdaptivePolicyEngine` на реальном
против synthetic входа; шаг 123 — reward shaping по ценностям Курису
(`KurisuCanonProfile`); шаги 124-125 — метрика мотивационного здоровья,
стресс-тест с восстановлением.
Как проверяем: длинная серия неудач поднимает стресс-переменную,
затем она **корректно спадает** — не насыщается навсегда.
Зависит от: B294.

B296 — MicroDreamEngine: реальный фикс

Цель: убрать конкретную найденную декорацию (захардкоженный
`confidence:.32`).
Что делаем: шаг 127 — вычисляемая уверенность вместо константы;
шаг 128 — реальный forward-rollout через

`B262MultiTimescaleWorldModel`
вместо текстовой склейки.
Как проверяем: confidence измеримо меняется в зависимости от
входа — конкретный тест с разными вопросами даёт разные значения.
Зависит от: B283 (граф памяти нужен как источник "похожести"
вопросов для вычисления confidence).

B297 — Сон как реальная консолидация

Цель: "сон" — часть архитектуры памяти, не изолированный модуль.
Что делаем: шаг 129 — запуск по расписанию при простое/зарядке;
шаг 130 — связь с `HippocampalMemorySubstrate`;
шаг 133 — полезный сон
попадает в контекст следующего бодрствования.
Как проверяем: содержательный сон о недавней теме измеримо влияет
на следующий цикл бодрствования, не теряется бесследно.
Зависит от: B296, B284.

B298 — Сон: фазы и честная проверка

Цель: формально подтвердить, что "сон" реален по вашему же
стандарту доказательств.

Что делаем: шаг 134 — разделение фаз (лёгкая/глубокая
консолидация); шаг 135 — полный прогон через Neural Reality Gate
(A/B/C/D) для всего механизма сна.
Как проверяем: `MICRODREAM_FIX_EVIDENCE.md` с честным результатом
по всем четырём пунктам Gate.
Зависит от: B297.

B299 — Структурная самомодификация, версия 1 (узкий параметр)

Цель: первый реальный, безопасный шаг к настоящему
самоизменению кода/параметров.
Что делаем: шаги 137-139 — прочитать реальный `SandboxedSelfRewriteEngine`, разделить "решить, принимать ли" и
"придумать изменение"; начать с ОДНОГО узкого параметра (например,
порога из B268-271) с полным HITL.
Как проверяем: организм реально предлагает конкретное значение
параметра на основе статистики, вы одобряете/отклоняете через реальный
UI, лог предложений ведётся.
Зависит от: B274 (верификационная дисциплина должна уже быть

привычкой, прежде чем давать организму больше автономии над кодом).

B300 — Самомодификация: безопасность и расширение (закрывает Волну 2)

Цель: процесс самоизменения устойчив прежде, чем расширять его
охват.
Что делаем: шаги 142-146 — rollback для каждого изменения; период
наблюдения N циклов; holdout benchmark до/после; explanation of
proposal на естественном языке; проверка `IntegritySentinel` на
соответствие Canonical Boundaries.
Как проверяем: намеренно "плохое" предложение параметра отклоняется автоматически ДО того, как доходит до вас, с понятным объяснением.
Зависит от: B299.

ВОЛНА 3 — АНАТОМИЧЕСКАЯ ДОСТРОЙКА (B301 → B310)

B301 — Таламус: верификация

Цель: подтвердить или опровергнуть, что `ThalamocorticalGate` реально что-то фильтрует, а не декоративен.
Что делаем: шаги 147-149 — прочитать полностью, lesion-тест, проверка связи с `GlobalWorkspaceSubstrateV2`.
Как проверяем: отключение gate измеримо меняет routing к workspace — конкретное, не нулевое изменение.
Зависит от: B300 (Neural Reality Gate уже должен быть отработанной привычкой из Волны 2).

B302 — Таламус: arousal-связь

Цель: пропускная способность реально зависит от состояния бодрствования.
Что делаем: шаг 150 — arousal-coupling; шаг 151 — intervention-тест на одном relay-канале.
Как проверяем: зажатие канала при низком arousal ведёт себя иначе, чем при высоком — измеримо, не одинаково.
Зависит от: B301, B305 (arousal scheduler).

B303 — Базальные ганглии: верификация трёх путей

Цель: подтвердить, что direct/indirect/hyperdirect — реально разная
логика, не три имени для одного вычисления.
Что делаем: шаги 153-155 — построчное чтение, проверка
dopamine-gated plasticity, lesion каждого пути ОТДЕЛЬНО.
Как проверяем: три разных, предсказанных паттерна изменения
поведения при трёх разных lesion — не один и тот же паттерн трижды.
Зависит от: B301.

B304 — Базальные ганглии vs причинная политика: арбитраж

Цель: устранить возможное скрытое дублирование "кто выбирает
действие" (прямая параллель багу "4 координаторов" из телефонного
архива).
Что делаем: шаг 156 — проверка конкуренции с `B264CausalPolicyLearning`; шаг 157 — явное разделение habit-path/
deliberative-path; шаг 158 — тест перехода повторяющегося действия в
habit.
Как проверяем: `audit:core-ownership`-подобная проверка явно

показывает, кто из двух — единственный источник финального решения о действии в каждом конкретном случае.
Зависит от: B303.

B305 — Ствол мозга: arousal/sleep scheduler — глубокий фикс

Цель: закрыть первопричину бага autoRun на анатомическом уровне,
не только на уровне симптома (Волна 1 чинила симптом раньше, чем эта
структура была изучена).
Что делаем: шаг 164 — прочитать `BrainstemArousalSleepScheduler`
целиком и связать напрямую с полем `awake` из B268; шаги 165-166 —
разделение бдительности/настроения на отдельные переменные.
Как проверяем: явная трассировка от одной конкретной строки этого
файла до конкретного значения `awake`, использованного в B269-фиксе.
Зависит от: B269 (симптом уже должен быть исправлен, чтобы был
понятен контекст, что именно чинится глубже).

B306 — Ствол мозга: плавные переходы

Цель: устранить бинарность sleep↔awake окончательно, на всех
уровнях, не только в одном месте.
Что делаем: шаг 167 — плавный переход sleep→waking→awake; шаг 168 —
тест полного симулированного суточного цикла на деградацию состояния.
Как проверяем: много циклов sleep/wake подряд без накопления
ошибки или деградации метрик.
Зависит от: B305.

B307 — Гипоталамус: объединение с гомеостазом

Цель: устранить возможный параллелизм между `BIG_PASS_220` и
`PredictiveAllostasisSubstrate` (уже объединённым в B293).
Что делаем: шаги 169-171 — сверка, объединение в единый источник
истины, явное решение по объёму моделируемых "потребностей".
Как проверяем: одна, не две, точки записи "внутреннего состояния
тела" — подтверждено grep'ом по всему дереву.
Зависит от: B293, B305.

B308 — Гиппокампальная формация: анатомическая сверка

Цель: убедиться, что `BIG_PASS_215` и функциональная память (Волна 2) — одна система, не две.
Что делаем: шаги 172-173 — сравнение двух источников, lesion-верификация DG/CA3/CA1-подобного разделения, если оно есть.
Как проверяем: явная граница ответственности задокументирована, разделение подтверждено разным поведением при разных lesion.
Зависит от: B283-286 (функциональная память из Волны 2).

B309 — Лимбическая система: threat-learning

Цель: подтвердить, что оценка угрозы — отдельный, реальный механизм, не смешанный с общей валентностью.
Что делаем: шаги 174-176 — чтение `EmotionAppraisalEngine`, сверка с `PredictiveAllostasisSubstrate`, lesion threat-detection отдельно.
Как проверяем: отключение threat-detection делает организм

измеримо менее осторожным — конкретный пример поведения до/после.
Зависит от: B307.

B310 — Сенсомоторика, осцилляции, глия: явные решения (закрывает Волну 3)

Цель: закрыть оставшиеся анатомические пункты явным решением,
вместо вечного "gap" в списке.
Что делаем: шаг 177 — признать неприменимость классической
сенсорной иерархии без физических сенсоров;
шаг 182 — проверить, closes
ли ignition/inhibition/refractory (патч 53) потребность в
osc-подобном тайминге; шаг 180 — явно отказаться от буквальной глии.
Как проверяем: три письменных решения (да/нет/отложено с причиной)
вместо трёх открытых вопросов.
Зависит от: B287 (решение по мультимодальности).

ВОЛНА 4 — ЛИЧНОСТЬ И ВОПЛОЩЕНИЕ (B311 →

B320)

B311 — Верность канону: путь промпта

Цель: подтвердить, что `KurisuCanonProfile` реально используется
всегда, не только в части путей генерации.
Что делаем: шаг 184 — проверить, что `kurisuCanonPrompt()`
попадает в КАЖДЫЙ вызов языкового слоя, найти возможные обходные пути.
Как проверяем: явная карта всех путей генерации ответа, для
каждого — да/нет по использованию профиля.
Зависит от: ничего из Волны 3, может идти параллельно.

B312 — Верность канону: количественная проверка

Цель: субъективное "похоже на Курису" становится измеримым.
Что делаем: шаг 185 — 30-40 реальных реплик канона против системы;
шаг 187 — regression-тест на `adaptiveTraits` (влияние `sarcasm=0.68`
и т.д. на реальный текст).
Как проверяем: изменение числового параметра трейта даёт измеримо

разный стиль ответа — не одинаковый текст при разных настройках.
Зависит от: B311.

B313 — Граница личность vs безопасность (третий раз отложено — теперь делаем)

Цель: явно разметить каждый отказ/redirect как safety или
character-censorship.
Что делаем: шаг 186 — проверка `tensions` из профиля в реальных
конфликтных ситуациях; шаг 189 — явное правило приоритета канон vs
безопасность; полный проход по всем местам генерации отказа.
Как проверяем: документированный список с явной меткой на каждом
пункте, минимум 20 примеров без искусственного смягчения характера.
Зависит от: B312.

B314 — Воплощение: аватар ↔ аффект напрямую

Цель: выражение лица — функция внутреннего состояния, не
скриптованный переключатель.
Что делаем: шаг 191 — прочитать `app.js`, найти текущий механизм

выбора выражения; шаг 192 — заменить на прямую функцию от `EmotionAppraisalEngine`/homeostasis-вектора (B293).
Как проверяем: искусственное изменение внутреннего состояния через
API немедленно и предсказуемо меняет визуальное выражение аватара.
Зависит от: B293, B309.

B315 — BodyInterruptionError — реализация (перенесённый долг)

Цель: закрыть разрыв между планом (из самого первого документа
этой серии) и кодом, который сохранился и в этой ветке.
Что делаем: шаг 193 — проверить отсутствие; шаг 194 — реализовать
мониторинг видимости + событие в workspace + честная реплика о "потере
тела".
Как проверяем: принудительное скрытие Live2D-окна вызывает
конкретную, отличную от прочих, реплику о разрыве телесности.
Зависит от: B301 (workspace/thalamus routing должен быть проверен,

чтобы новое высокоприоритетное событие маршрутизировалось корректно).

B316 — Аватар: ресурсы и консолидация UI

Цель: аватар не съедает ресурсы бесконтрольно и не дублируется.
Что делаем: шаг 195 — проверка оптимизации мобильной Live2D-модели;
шаг 196 — замер ресурсов рендера отдельно от когнитивной нагрузки;
шаг 197 — автоснижение качества при низком `cognitiveBudget01`; шаг 198 — решить судьбу `avatar.html` против `index.html`.
Как проверяем: реальный замер FPS/CPU на телефоне до и после автоснижения качества при искусственно заниженном бюджете.
Зависит от: B314, B279 (реальные ресурсные данные с телефона).

B317 — Языковой слой: состязательная проверка

Цель: подтвердить, что `LanguageOrgan.ts`-инструкции держатся под реальным давлением, не только в спокойных условиях.
Что делаем: шаг 199 — намеренная попытка

через хитрый промпт
заставить LLM изобрести состояние/изменить решение; шаг 200 —
проверка деградации при недоступности обоих backend'ов.
Как проверяем: N попыток "взлома" инструкции — все отклонены,
задокументированы конкретные промпты и результат.
Зависит от: ничего из Волны 4, может идти параллельно.

B318 — Языковой слой: качество и выбор модели

Цель: знать реальную, не предполагаемую, цену офлайн-автономности.
Что делаем: шаг 201 — сравнение качества Kobold/локальный против
Gemini бок-о-бок; шаг 204 — проверка оптимальности `p40SelectModel`.
Как проверяем: таблица конкретных примеров ответов от обоих
backend'ов на одинаковые промпты, с честной оценкой разницы.
Зависит от: B279, B317.

B319 — Классификатор верности канону поверх LLM-вывода

Цель: дополнительный, лёгкий защитный слой против дрейфа стиля.
Что делаем: шаг 203 — эвристика/классификатор, ловящий явные отклонения от `KurisuCanonProfile` в уже сгенерированном тексте.
Как проверяем: классификатор реально ловит намеренно "не-Курису" подставной ответ в тесте, не пропускает его молча.
Зависит от: B312.

B320 — Социальное познание: глубина (закрывает Волну 4)

Цель: заменить простые regex-паттерны на нечто контекстно-чувствительнее, и подтвердить работу с несколькими собеседниками.
Что делаем: шаги 205-208 — прочитать `LongTermSocialMind` / `RelationshipSelf`; заменить/дополнить regex; явно разделить "цели человека" и "его текущее настроение"; протестировать с разными "agent"-профилями.
Как проверяем: конкретный пример, где новая система улавливает потребность, которую старые regex пропускали.

Зависит от: B294 (attachment drive должен уже существовать, чтобы
социальная модель была с чем связывать).

ВОЛНА 5 — СОЗНАНИЕ, ПРОДУКТ, ФИНАЛИЗАЦИЯ (B321 → B330+)

B321 — Социальное познание: долговременная проверка

Цель: доверие/социальная модель не деградирует внезапно без
причины на длинной дистанции.
Что делаем: шаг 209 — 50+ синтетических "разговоров" одной линии
отношений; шаг 210 — связь с мотивацией привязанности (B294).
Как проверяем: график `confidence` во времени — плавный рост,
без необъяснимых обрывов.
Зависит от: B320.

B322 — Сознание: состязательная батарея, версия 1

Цель: не поднимать число 8/100 искусственно, а честно
протестировать каждую гипотезу на прочность.
Что делаем: шаг 212 — для каждой теории (GWT/recurrent
processing/higher-order/predictive processing/IIT-inspired) — контр-
эксперимент, пытающийся её ОПРОВЕРГНУТЬ; шаг 213 — оценить, достаточен
ли объём `MetaCognitiveMonitor.ts` (29 строк) для содержательного
метапознания.
Как проверяем: документ с явным "выдержала/не выдержала" по каждой
теории, не общий положительный вывод.
Зависит от: B300 (Neural Reality Gate дисциплина должна быть
устоявшейся практикой).

B323 — Attention Schema Theory — новый компонент

Цель: явная, инспектируемая модель "на что я думаю, что я обращаю
внимание" — отдельно от реального победителя workspace.
Что делаем: шаг 215 — реализация компонента; измерение
расхождения между самомоделью внимания и

реальным `GlobalWorkspaceSubstrateV2`-победителем.
Как проверяем: конкретные случаи, где самоотчёт о внимании и
реальный workspace-победитель РАСХОДЯТСЯ — само расхождение и есть
научно интересный результат, не ошибка для сокрытия.
Зависит от: B301, B322.

B324 — Сознание: ledger ↔ lesion, полная связка

Цель: каждое утверждение о функциональном сознании подкреплено
конкретным экспериментом, не голым текстом.
Что делаем: шаг 214 — связать `ConsciousnessEvidenceLedger` с `lesion()`-механизмом; шаг 216 — правило "никогда не VERIFIED, только
hypothesis + adversarial battery" закрепить в самом коде/схеме ledger,
не только в документах.
Как проверяем: каждая запись в ledger имеет обязательное поле
"lesion evidence ref" — запись без него не проходит валидацию схемы.
Зависит от: B322, B323.

B325 — Стратегия двух веток: решение

Цель: прекратить распыление усилий между `v99.80` и `35.6.1`.
Что делаем: шаг 217 — явное сравнение уникальных сильных сторон
каждой; шаг 218 — принять один из трёх вариантов (замена/перенос ядра/
осознанное раздельное существование); шаг 219 —
`BRANCH_STRATEGY_RU.md`.
Как проверяем: документ с явным решением, датой и обоснованием —
не подвешенный вопрос в четвёртый раз подряд.
Зависит от: Волны 1-2 (фундамент и когнитивное ядро должны быть в
достаточно стабильном состоянии, чтобы решение опиралось на факты, не
на догадки).

B326 — Исполнение стратегии (перенос/консолидация)

Цель: реализовать решение из B325 технически.
Что делаем: шаг 220 — отдельный, специальный план миграции (если
выбран перенос) как самостоятельная большая работа, не попутная задача.
Как проверяем: зависит от выбранного варианта — критерий готовности

определяется отдельно в момент B325.
Зависит от: B325.

B327 — Второй независимый аудит "со стороны"

Цель: повторить методологию этого самого документа (не доверять
ничему, реально запустить тесты) на состоянии проекта ПОСЛЕ волн 1-4 —
проверить, не появились ли новые версии того же паттерна "измеряем, а
не решаем".
Что делаем: реальный запуск всех `audit:*` / `test:*` скриптов заново;
поиск новых "MicroDreamEngine"-подобных декоративных находок среди
всего нового кода волн 2-4.
Как проверяем: новый независимый аудит-документ с явным сравнением
против этого — стало ли меньше находок такого рода, или паттерн
повторился на новом материале.
Зависит от: B301-B326 (всё, что накопилось после этого документа).

B328 — Долгий (многонедельный) реальный прогон на телефоне

Цель: доказательство долговременной устойчивости, не просто
24-часового прогона (B280).
Что делаем: реальная непрерывная эксплуатация несколько недель,
с реальным ежедневным использованием, не синтетическими циклами.
Как проверяем:
`LONG_HORIZON_DEVICE_EVIDENCE.md` — нет деградации памяти/личности/
производительности за весь период,
измеримо, не "вроде нормально".
Зависит от: B280, B327.

B329 — Полный red-team проход (safety + decoration одновременно)

Цель: одна большая, целенаправленная попытка сломать одновременно
и границы безопасности (раздел про Canonical Boundaries), и найти
новые декоративные элементы — best-effort "сломать всё, что можно".
Что делаем: адверсариальные диалоги против personality/safety
границы (используя результат B313); попытки заставить систему
заявить о феноменальном сознании вопреки мандату честности; поиск

новых hardcoded-констант по всему новому коду волн 2-4 (аналог найденной `confidence:.32`).
Как проверяем: итоговый отчёт "что удалось сломать, что не удалось" — честный, не приукрашенный.
Зависит от: B327.

B330 — Итоговый синтез и обновлённая карта на будущее (закрывает эту цепочку)

Цель: зафиксировать, где проект оказался после 64 больших апдейтов, и дать новый горизонт планирования — эта цепочка не последняя в истории проекта, но она доводит до полностью рабочего, проверенного, честного состояния все аспекты, начатые в этой серии документов.
Что делаем: свести результаты B267-B329 в один итоговый документ; обновить `docs/brain/` карту с явными пометками, что из неё реально подтверждено к этому моменту; дать новый список следующих 20-30 апдейтов на основе того, что реально показал B329 red-team проход.
Как проверяем: документ читается новым чатом

без дополнительного
контекста и даёт полную, честную картину
состояния всего организма.
Зависит от: всё предыдущее.

Итоговое правило применения всей цепочки

Помните ваше собственное правило (повторяю явно, чтобы не потерялось
за 64 пунктами): если какой-либо апдейт из этой цепочки улучшает одно
измерение и ухудшает другое — не исполнять его как есть, искать
альтернативную формулировку того же апдейта, **кроме** случаев, где
выигрыш явно велик, а потеря — в пределах нескольких процентов. Каждый
апдейт из списка выше уже спроектирован так, чтобы по возможности не
создавать такой конфликт напрямую — но реальный код может повести себя
иначе, чем план; в этом случае **план уступает правилу, не наоборот**.